

Lesson 9.2 — Demo Runbook (Laravel-only, с конкретным кодом)

Этот файл — единая инструкция для мита: **слайды -> этот runbook -> обратно к слайдам**.
Во время демо `lesson_9_2_practice.md` не открываем.

0) Проверка соответствия презентации

По презентации `lesson_9_2_presentation.pdf` практические переходы идут в этих точках:

- **Слайд 6/25:** `CurrencyService` с кешем + `invalidate()`
- **Слайд 16/25:** `PaymentRepository` без N+1
- **Слайды 21–22/25:** задания на анализ endpoint-ов, N+1, индексы, измерения

В этом runbook эти переходы покрыты блоками:

- Блок А -> слайд 6 (кеш + инвалидация)
- Блок В -> слайд 16 (with + пагинация)
- Блок С/D -> слайды 21–22 (измерения + индексы)

Важно: в текущем runtime-коде проекта `CurrencyService` может отсутствовать — для урока показываем его как **учебный точечный пример** (без ломки текущей реализации).

1) Pre-flight (до начала лекции)

1. Проверить контейнеры:
 - `docker compose ps`
2. Проверить роуты:
 - `docker compose exec -T php php artisan route:list --path=api/v1`
3. Проверить тесты на SQLite:
 - `docker compose exec -T php php artisan test tests/Feature/Api/V1`

Установка Telescope (делаем один раз в проекте)

Если `http://localhost/telescope/requests` не открывается или `route:list --path=telescope` пустой:

1. Установить пакет:
 - `docker compose exec -T php composer require laravel/telescope --dev`
2. Опубликовать конфиг и миграции:
 - `docker compose exec -T php php artisan telescope:install`
3. Применить миграции:
 - `docker compose exec -T php php artisan migrate --force`
4. Очистить кеш:
 - `docker compose exec -T php php artisan optimize:clear`
5. Проверить маршруты Telescope:

- `docker compose exec -T php php artisan route:list --path=telescope`

6. Открыть UI:

- `http://localhost/telescope/requests`

Примечание: в этом проекте базовый URL при `docker compose` — `http://localhost`, не `http://laravel.test`.

Установка Debugbar (опционально, для быстрых метрик в браузере)

Если хочешь показывать SQL-метрики прямо на странице endpoint-a:

1. Установить пакет:

- `docker compose exec -T php composer require barryvdh/laravel-debugbar --dev`

2. Очистить кеш:

- `docker compose exec -T php php artisan optimize:clear`

3. Проверить, что в `.env` есть:

- `APP_DEBUG=true`

4. Открыть endpoint в браузере (не через curl), например:

- `http://localhost/api/v1/accounts/1/payments?per_page=20`

5. В нижней панели Debugbar открыть вкладку `Queries` и показать:

- количество SQL-запросов;
- время каждого запроса;
- дубликаты запросов (часто индикатор N+1).

Как удалить Telescope и Debugbar после урока (если нужно)

Удаление Telescope

1. Удалить пакет:

- `docker compose exec -T php composer remove laravel/telescope --dev`

2. Удалить таблицы Telescope (через rollback именно его миграции или вручную, если нужно):

- `docker compose exec -T php php artisan migrate:status`
- если нужно быстро вручную:
 - `docker compose exec -T mysql mysql -ufinance_user -pfinance_password finance_db -e "DROP TABLE IF EXISTS telescope_entries_tags; DROP TABLE IF EXISTS telescope_entries; DROP TABLE IF EXISTS telescope_monitoring;"`

3. Очистить кеш:

- `docker compose exec -T php php artisan optimize:clear`

4. Проверить, что роутов Telescope больше нет:

- `docker compose exec -T php php artisan route:list --path=telescope`

Удаление Debugbar

1. Удалить пакет:

- `docker compose exec -T php composer remove barryvdh/laravel-debugbar --dev`

2. Очистить кеш:

- `docker compose exec -T php php artisan optimize:clear`

3. В `.env` при необходимости вернуть:

- `APP_DEBUG=false`

2) Блок А — кеш + инвалидация (переход со слайда 6)

2.1 Что говорить

- Кешируем то, что редко меняется: справочники/настройки.
- Обязательно показываем **инвалидацию** (иначе будет stale data).
- Для финансовых real-time данных (баланс перед списанием) кеш без строгой стратегии опасен.

2.2 Код для демонстрации (новый сервис в Laravel)

Файл для примера: `app/Services/CurrencyService.php`

```
<?php

declare(strict_types=1);

namespace App\Services;

use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Collection;

final class CurrencyService
{
    private const CACHE_KEY = 'currencies:list';
    private const TTL_SECONDS = 3600;

    /**
     * @return Collection<int, string>
     */
    public function getAll(): Collection
    {
        return Cache::remember(self::CACHE_KEY, self::TTL_SECONDS,
function (): Collection {
            // Уроковый пример: в проекте можно заменить на
            Currency::query()->orderBy('code')->pluck('code')
            return collect(['USD', 'EUR', 'UAH']);
        });
    }

    public function invalidate(): void
    {
        Cache::forget(self::CACHE_KEY);
    }
}
```

2.3 Примеры инвалидации кеша (что сказать и где делать)

1. После изменения справочника валют (админка):

```
$currencyService->invalidate();
```

2. После создания/изменения платежа (инвалидация dashboard-кеша):

```
Cache::forget("accounts:{$accountId}:payments:latest:10");  
Cache::forget("accounts:{$accountId}:payments:summary:current_month");
```

3. Tag-based (если используете Redis tags):

```
Cache::tags(["account:{$accountId}", 'payments'])->flush();
```

3) Блок В — N+1 и `with()` (переход со слайда 16)

3.1 Что открыть в проекте

- `app/Repositories/PaymentRepository.php`
- метод `paginateByAccountId(...)`

Базовый фрагмент:

```
Payment::query()  
    ->where('account_id', $accountId)  
    ->with('account')  
    ->latest()  
    ->paginate($perPage);
```

3.2 Что рассказать про `with()` (расширенно)

1. Одна связь:

```
->with('account')
```

2. Несколько связей:

```
->with(['account', 'user'])
```

3. Вложенные связи:

```
->with(['account.user'])
```

4. С условиями на связь (closure constraints):

```
->with([
  'payments' => function ($query) {
    $query->where('status', 'processed')->latest();
  },
])
```

5. Выбор нужных полей связи (снижение объема):

```
->with(['account:id,balance', 'user:id,email'])
```

6. Lazy eager loading после запроса (когда коллекция уже получена):

```
$payments = Payment::query()->where('account_id', $accountId)->get();
$payments->load('account');
```

7. `withCount()` для счетчиков без загрузки всей коллекции:

```
Account::query()->withCount('payments')->get();
```

Ключевая мысль для мита: `with()` фиксирует количество запросов и убирает 1+N паттерн на списках.

4) Блок С — метрика через Telescope/Debugbar (добавляем в демонстрацию)

4.1 Что измеряем студентам

Для endpoint `GET /api/v1/accounts/{account}/payments` показываем 3 метрики:

1. `SQL queries count` — сколько SQL-запросов на один HTTP-запрос.
2. `Total query time (ms)` — суммарное время SQL.
3. `Slowest query (ms)` — самый медленный запрос.

Это ровно то, что нужно для цикла «измерили -> изменили -> измерили снова».

4.2 Как показать через Telescope

1. Открыть `http://localhost/telescope/requests` (или `queries`).
2. Выполнить в терминале:
 - `curl -s "http://localhost/api/v1/accounts/ID/payments?per_page=20" > /dev/null`
3. В Telescope выбрать этот request и показать:
 - сколько SQL;
 - общее время запросов;
 - самый тяжелый SQL.
4. После изменения (например, eager loading/индекс) повторить тот же curl и сравнить.

4.3 Если используешь Debugbar вместо Telescope

1. Открыть endpoint в браузере.
2. Вкладка **Queries**:
 - число запросов;
 - время каждого;
 - повторы (признак N+1).
3. Зафиксировать цифры до/после.

4.4 Мини-шаблон фиксации результатов (вслух или на доске)

- До: `queries = X, total_sql_ms = Y, slowest_ms = Z`
 - После: `queries = X2, total_sql_ms = Y2, slowest_ms = Z2`
 - Вывод: что улучшилось и за счет чего.
-

5) Блок D — измеримый кейс индексов через API endpoint (без «чистого SQL» как основного фокуса)

5.1 Генерация больших данных через фабрики (чтобы разница была ощутимой)

Перед сравнением подготовь данные:

1. Создай тестовый аккаунт:
 - `curl -s -X POST http://localhost/api/v1/accounts -H "Content-Type: application/json" -d '{"balance":"100000.00"}'`
2. Возьми `id` аккаунта.
3. Наполни `payments` большим количеством записей через Tinker в контейнере:
 - `docker compose exec -T php php artisan tinker -- execute="\App\Models\Payment::factory()->count(20000)->create(['account_id' => ID, 'status' => 'processed', 'currency' => 'USD']);"`

5.2 Что именно меряем через endpoint

Endpoint для замера:

- `GET /api/v1/accounts/{ID}/payments?per_page=50`

Сценарий:

1. Открой Telescope (`/telescope/requests`).
2. Выполни 3-5 одинаковых запросов endpoint-a:
 - `for i in {1..5}; do curl -s "http://localhost/api/v1/accounts/ID/payments?per_page=50" > /dev/null; done`
3. Зафиксируй:
 - `total request time`;
 - `queries count`;
 - `slowest query time`.

5.3 Добавить индекс (пример миграции)

```
Schema::table('payments', function (Blueprint $table): void {  
    $table->index(['account_id', 'created_at'],  
    'payments_account_created_idx');  
});
```

5.4 Измерить после индекса (тем же endpoint-ом)

1. Примени миграцию с индексом.
2. Повтори те же 3-5 запросов endpoint-a.
3. Сравни метрики до/после:
 - `request time` ниже;
 - `slowest query time` ниже;
 - при больших данных разница заметна даже визуально в Telescope.

Ключевая мысль для студентов:

- мы измеряем реальный API-сценарий пользователя, а не искусственный SQL в вакууме.

6) Блок Е — Live endpoint сценарий

1. Создать аккаунт:
 - `curl -s -X POST http://localhost/api/v1/accounts -H "Content-Type: application/json" -d '{"balance":"5000.00"}'`
2. Взять `id` из JSON.
3. Получить список платежей:
 - `curl -s "http://localhost/api/v1/accounts/ID/payments?per_page=20"`
4. Показать:
 - `data`, `links`, `meta` (пагинация)

7) Что уже реализовано и что не трогаем

Уже есть и работает:

- `GET /api/v1/accounts/{account}/payments`
- FormRequest + DTO + Service + Repository + Feature tests

Это оставляем как есть.

`CurrencyService` демонстрируется как учебный пример из слайдов, без обязательного внедрения в runtime-поток.

8) Тесты и логи в конце демо

1. Целевой тест endpoint-a:
 - `docker compose exec -T php php artisan test tests/Feature/Api/V1/ListAccountPaymentsTest.php`
 2. Полный API набор:
 - `docker compose exec -T php php artisan test tests/Feature/Api/V1`
 3. Логи:
 - `docker compose exec -T php sh -lc 'tail -n 100 storage/logs/laravel.log'`
-

9) Anti-fail (30 секунд до старта)

1. `docker compose exec -T php php artisan config:clear`
2. `docker compose exec -T php php artisan route:list --path=api/v1/accounts`
3. `docker compose exec -T php php artisan test tests/Feature/Api/V1/ListAccountPaymentsTest.php`